

This document will explain everything you need to know in order to develop your CPU in the lab with SystemVerilog. If you need a refresh on SystemVerilog take a look at both Rechenanlagen lectures on the topic ([Part 1](#), [Part 2](#)). Assembling your programs to binaries and programming your CPU will be covered in another document later. We assume that you have a basic understanding of [git](#), as you will organise your project with it. [Here](#) you can find a simple guide, if you never heard of it before. You should know how to add, commit and push. Otherwise just ask, we will help you.

We'll start with the installation of the toolchain, if you don't plan on using the computers in the lab. Then we will step through the entire toolchain with a simple example module that should help you get started with hardware description and FPGA synthesis. Finally, we will look at the most relevant cells featured in our FPGA.

Contents

1	Installation	1
2	Using the toolchain	2
2.1	Preparation	3
2.2	Step: Simulation (sim)	4
2.3	Step: Synthesis (synth)	6
2.4	Step: Place & Route (pr)	7
2.5	Step: Generate Bitstream (gen)	9
2.6	Step: Flash Bitstream (flash)	9
2.7	Summary	9
3	ICE40 cells	10
3.1	SB_LUT4	10
3.2	SB_DFF	11
3.3	SB_RAM	11
3.4	SB_MAC16	12
3.5	SB_IO	13
3.6	SB_SPRAM	13
3.7	SB_CARRY	13
3.8	SB_GB	14
4	All Makefile targets	14

1 Installation

It is recommended that you use the computers in the hardware lab as the toolchain is already installed there. If you want to use your own device or be able to work from home, then this chapter is for you. Otherwise you can skip it. We will assume you are either using a Unix based OS or WSL if you are on windows.

Most of the tools are bundled in [oss-cad-suite](#). Download a release suitable for your architecture, extract the archive and activate the environment to update your **PATH**. In the lab, you will find it installed at **/opt**. At the time of writing, the installation could look like this (replace **<...>** with your actual filenames):

```
$ cd <path-to-oss-cad-suite>
$ tar xvfz oss-cad-suite-<...>.tgz
$ source oss-cad-suite/environment # Repeat for every session or automate
```

Our logic analyzers can be used together with the official software from Saleae. It is distributed as an AppImage, therefore you should download the file from [here](#) and mark it as executable:

```
$ chmod u+x Logic-<your-exact-version>.AppImage # replace with your filename
$ ./Logic-<your-exact-version>.AppImage # run the program
```

[Surfer](#) is a more modern replacement for GTKWave, which is already bundled with oss-cad-suite. We recommend Surfer. You can get a binary from the website or compile the Rust crate yourself.

Optional: If you want to visualize your synthesis results, you need two npm packages:

```
$ npm install -g netlistsvg
$ npm install -g svgo
```

and `rsvg-convert`. Depending on your distribution this package may be called `librsvg` or `librsvg-utils`.

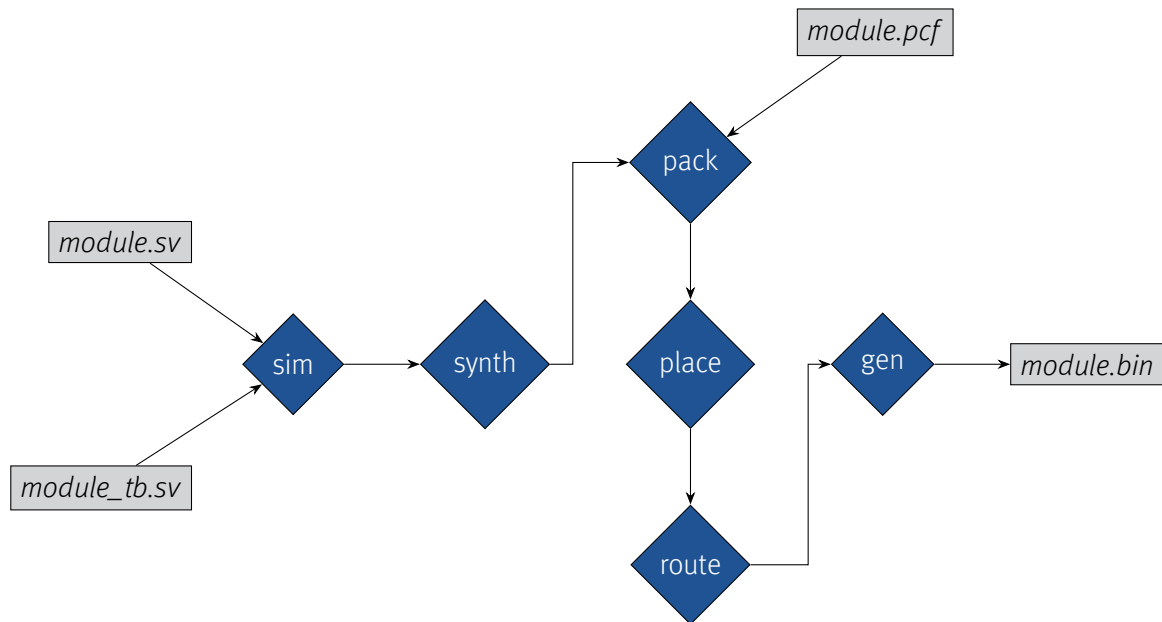
Optional: If you want to flash the FPGA and use the logic analyzer without root privileges, you need to install two udev rules, which are provided in the repo

```
# cp *.rules /etc/udev/rules.d/
# udevadm control --reload-rules
# udevadm trigger
```

Optional: Take a look at [direnv](#), if you want to automate activating the oss-cad-suite environment and at [verible](#) for a language server working with Emacs, Vim and VS Code

2 Using the toolchain

The development process in this lab consists of different stages, most of which are shown in the following picture (assembly excluded):

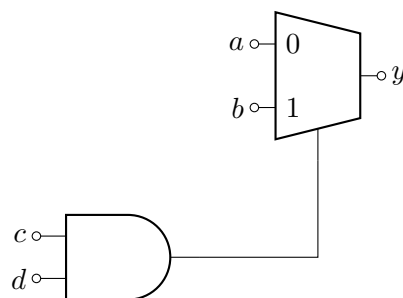


Let's examine the stages in more detail. You don't have to understand everything at the moment, this is mostly to give you a broader idea of what's happening with your design. Let's start with step 0 and actually write a small example module `ifelse`. It will follow us along the way:

```

1 module ifelse (
2     input logic a, b, c, d,
3     output logic y
4 );
5
6 always_comb begin
7     if (c == 1'b1 && d == 1'b1)
8         y = b;
9     else
10        y = a;
11 end
12 endmodule
  
```

Think about this module briefly. How would you build a circuit that implements the same behavior? We expect to see this:



2.1 Preparation

The basic project structure should look like this:

```

.
|-- Makefile.inc
  
```

```

`-- ifelse
  |-- Makefile
  `-- ifelse.sv

```

If you are at the root of the repository, then first cd into the rtl directory:

```
$ cd rtl
```

At the root is a Makefile called **Makefile.inc**, which is used by all modules. It contains all relevant information on how to perform the build process. Then we have a folder which contains our module and a second Makefile. The nested Makefile just sets some variables specific to this module (mostly the name). If you don't plan on adding your own modules, then you never have to touch a Makefile. For completeness, the Makefile here would have the following contents:

```

1  # Name of module, module.sv, module_tb.sv
2  FILE=ifelse
3
4  include ../Makefile.inc

```

Let's activate oss-cad-suite, so we have access to all tools and change directory to start the build process:

```

$ source /opt/oss-cad-suite/environment
$ cd ifelse # cd into rtl/ifelse

```

2.2 Step: Simulation (sim)

The first step after writing our module is simulating and testing it. We'll be using [Icarus Verilog](#) as our simulator. However, on its own our module is pretty useless. We have only defined how the output behaves depending on the inputs. To actually provide some inputs we must instantiate our module. This is typically done in a separate testbench. During the semester, most testbenches will be provided, but let's write a simple one right now:

```

1  `timescale 1ns/1ns
2
3  module ifelse_tb();
4      logic a, b, c, d, y;
5
6      ifelse dut (.a(a), .b(b), .c(c), .d(d), .y(y));
7
8      initial begin
9          $dumpfile("ifelse.vcd");
10         $dumpvars(0, dut);
11
12         a <= 0; b <= 0; c <= 0; d <= 0;
13         #10;
14         assert(y==a) else $error("y not equal a");
15
16         a <= 1; b <= 0; c <= 1; d <= 1;
17         #10;
18         assert(y==b) else $error("y not equal b");
19         $finish;

```

```
20     end
21 endmodule
```

This testbench is just a module without inputs. It creates an instance of `ifelse` and maps all inputs and outputs to signals in the testbench. In the following `initial` block we assign values to the signals and wait for 10 time units (in this case 1 ns), then change inputs and finally assert the output. After saving the file, our project looks like this:

```
.
|-- Makefile.inc
`-- ifelse
    |-- Makefile
    |-- ifelse.sv
    `-- ifelse_tb.sv
```

With everything in place, we can now run the simulation. Execute the »simulation« target by running

```
$ make simulation
```

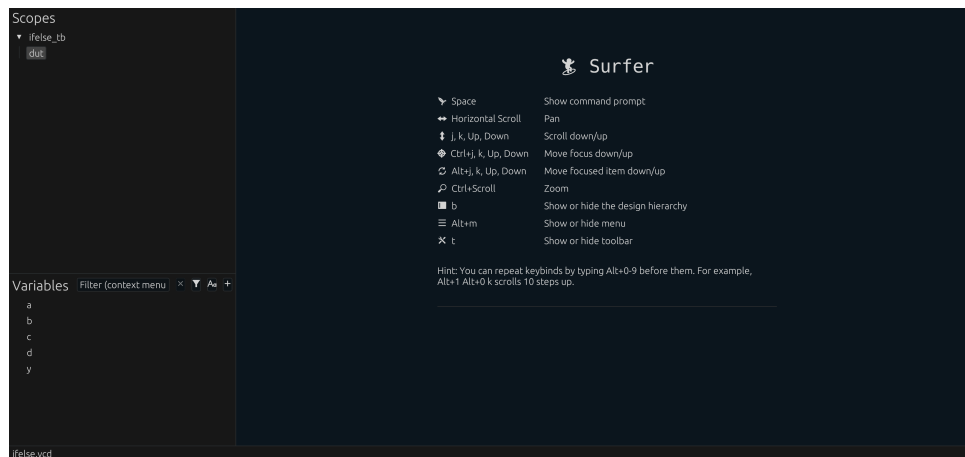
and Icarus Verilog returns the following output

```
VCD info: dumpfile ifelse.vcd opened for output.
ifelse_tb.sv:33: $finish called at 20 (1ns)
```

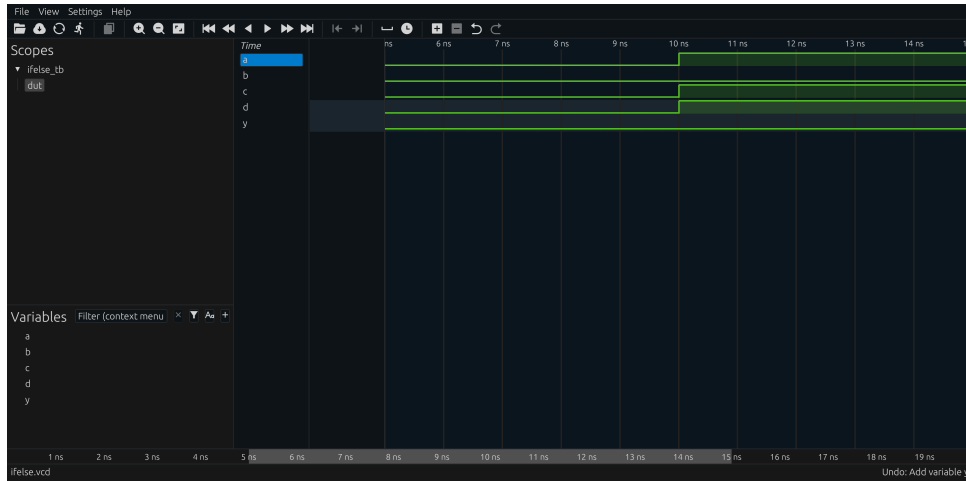
So no errors! We also dumped everything into a waveform file (because we used `$dumpfile`), which allows us to see the value of every signal at every point in time. We can use a waveform viewer to look at it. We will use Surfer here. Run it by invoking

```
$ surfer ifelse.vcd
```

which opens this window:



In the top left you can see the hierarchy of your project. We see our testbench as the root and the module instance `dut`, which we defined in our testbench. In the middle left we see all signals of the selected module. By double-clicking we can add their trace to the main window in the right. Examining the waveform will be your goto tool of finding bugs in your design!



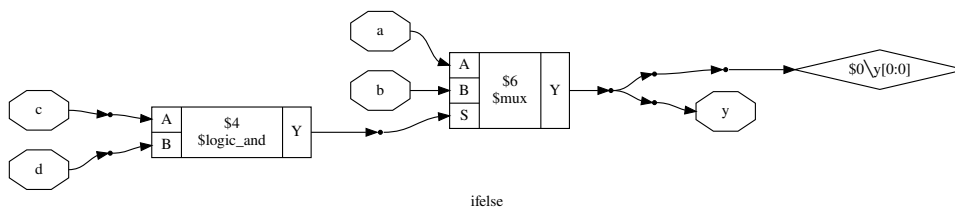
Looking good! Our module behaves like we want. Let's see if we can also synthesize it or if there are some problems.

2.3 Step: Synthesis (synth)

The synthesis stage just needs our module description (`ifelse.sv`) as an input. We will be using [Yosys](#) as our synthesis tool. Yosys produces the netlist `ifelse.json`, which consists of our module - expressed with whatever hardware components we have. This is highly dependant of the target architecture. Thinking back to the lectures these would be logic gates, flip-flops, latches, multiplexers, Yosys can perform such a generic synthesis, let's look at it:

```
$ make synthesis_generic
```

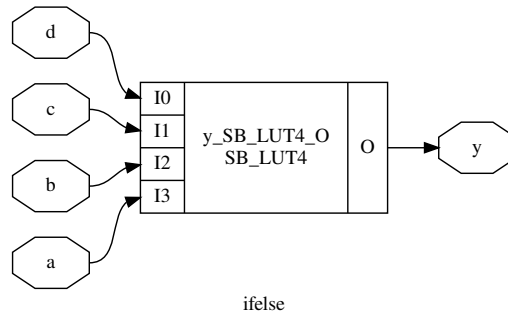
This produces lots of output, but it leaves us with the netlist `ifelse.json` and this visual representation:



We see the inputs, outputs and the logic between them. In this case exactly what we expected to see, an AND gate and a multiplexer for the output. If we change the architecture and actually synthesize for the ICE40 by running

```
$ make synthesis
```

we get a netlist that looks like this (visualization commented out by default; takes very, very long on actual designs later and is almost useless):



Remember how the result is highly dependant on the architecture? Our FPGA doesn't have logic gates or multiplexers. `SB_LUT4` is a lookup table with four inputs. Think of a truthtable with four variables. Our module is very simple and can actually be represented with just one truthtable, so Yosys does exactly that. It knows the cell library of the FPGA and tries to find the optimal implementation. Some other cells are explained in Section 3. There are more, but not all of them are useful for us here. With this, we finished the synthesis stage and can continue.

2.4 Step: Place & Route (pr)

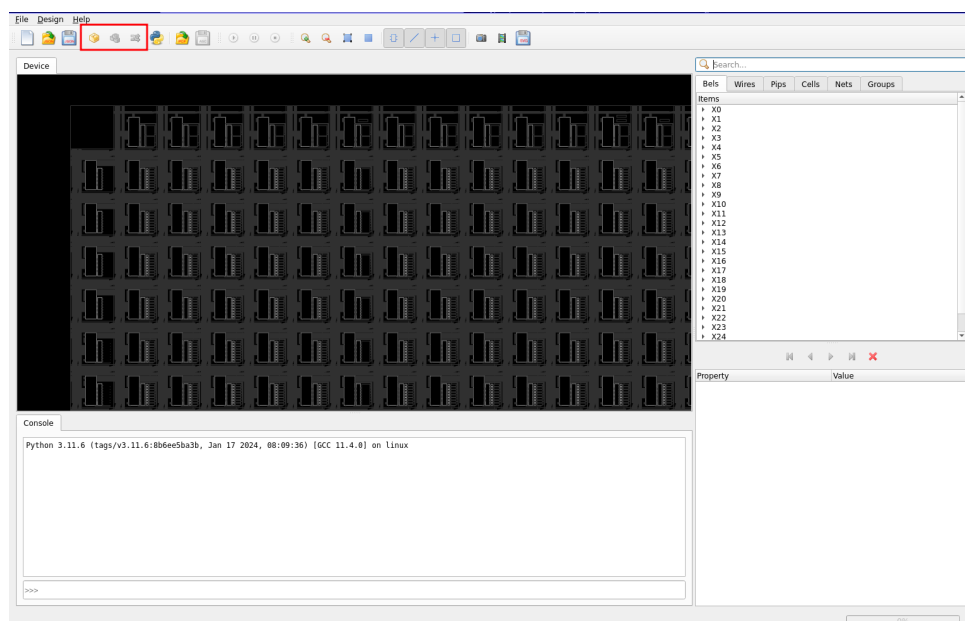
Our previously generated netlist `ifelse.json` contains a list of hardware components and their connections. However, our FPGA has thousands of `SB_LUT4` and wires between them. The goal of this stage is to map our netlist to the actual hardware on the FPGA. This means »placing« all components and »routing« their inputs and outputs to the FPGA pins specified in a `.pcf` file. In our toolchain, we are using `nextpnr` for this. We have two options of running `nextpnr`. Usually you will just run

```
$ make pr
```

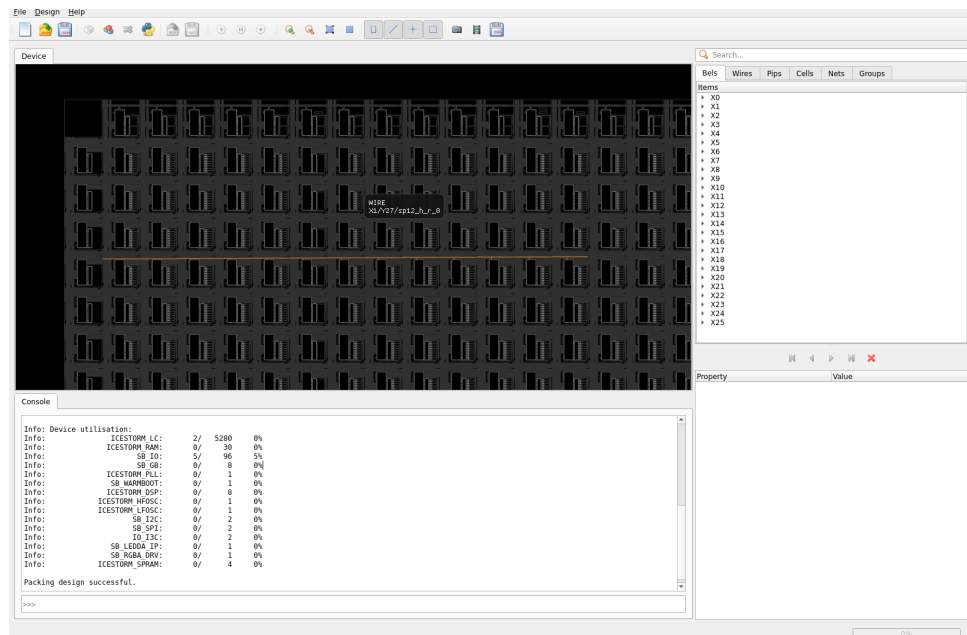
which gives us some output and the file `ifelse.asc` that we can use to continue the build process. But let's take a deeper look right now. By running

```
$ make pr-gui
```

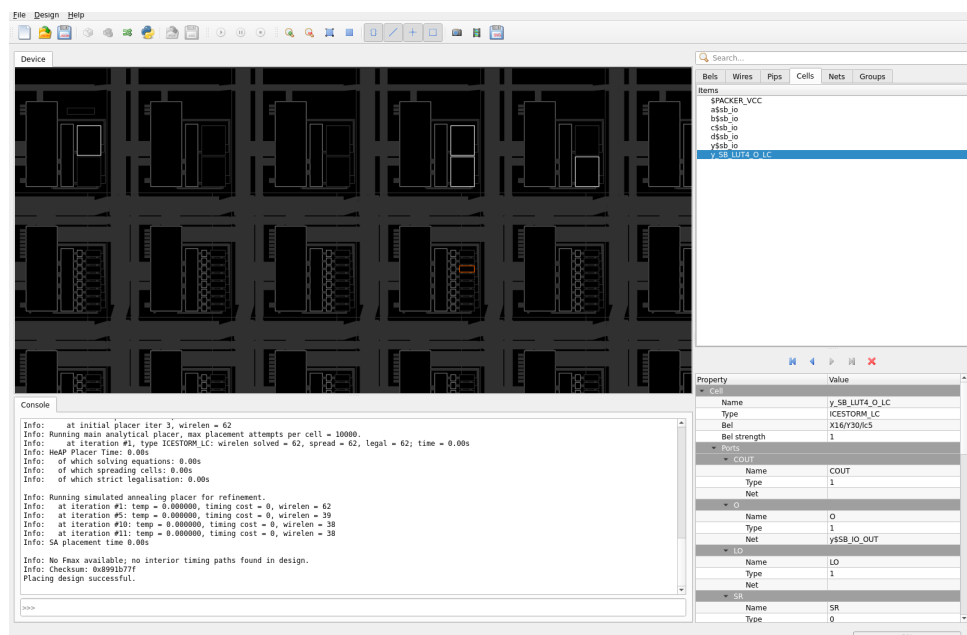
we can open `nextpnr` with its gui. This shows us the empty FPGA with its internal structure (zoomed in a bit, so we can actually see anything)



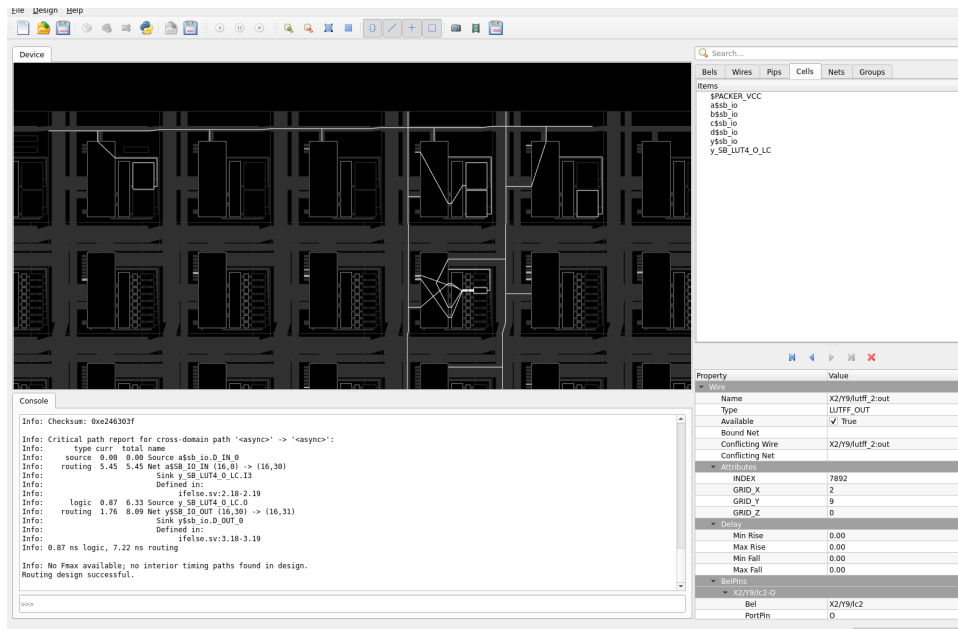
The three buttons in the top left allow us to perform the steps »pack«, »place« and »route« manually. Let's start with the first one:



Nothing changed in the FPGA structure, but we got some output telling us the usage statistics. Notice the 5 **SB_IO** cells. They are used to connect our design to the pins of the FPGA, so one gets packed for every IO signal we have. Additionally, our **SB_LUT4** has disappeared and nextpnr packed two **ICESTORM_LC** cells instead. Consider them equal for now. One of them implements the **SB_LUT4** from our module, the second one is called **PACKER_VCC** and will provide the constant 1 to other cells. Performing »place« gives us



which shows most of the previously packed cells now placed in specific locations on the FPGA. In this picture we can see the four inputs in the top and the lookup table from before in orange below. This step also performs a timing analysis, which would verify that we can run this design with our target clock frequency (we always use 12 MHz, although we don't have a clock here). Finally, routing leaves us with



which concludes this step. During this course, only a few modules can be placed and routed on their own. We have a very limited amount of IO ports and most modules will exceed them. You can still use nextpnr to check resource usage, but everything beyond will fail.

2.5 Step: Generate Bitstream (gen)

This step converts the routed design from a text representation in `ifelse.asc` to the binary `ifelse.bin`, which the FPGA actually understands. Invoke it by running

```
$ make gen_bitstream
```

2.6 Step: Flash Bitstream (flash)

Our build process is finished by now. Connect the FPGA and run

```
$ make flash_bitstream
```

which will send the bitstream binary to the FPGA and program the internal structure to match our design

2.7 Summary

This tutorial gave you way more insights into the toolchain than you will need for most of the time. Summing it up, your workflow will mainly consist of either simulating your design and testing it by running

```
$ make simulation
```

or by performing the entire FPGA build process. The Makefile knows how to perform everything for you, you can just invoke

```
$ make flash_bitstream
```

which will synthesize and place & route your design, generate the bitstream and flash the FPGA. If you want to delete all build artifacts you can use

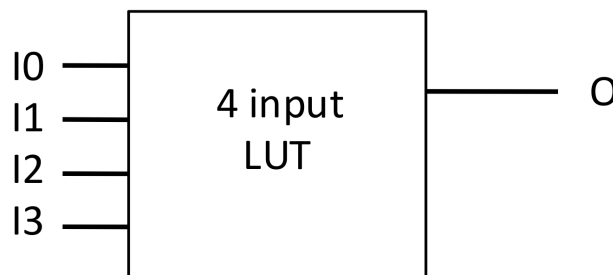
```
$ make clean
```

3 ICE40 cells

This chapter will look at most ICE40 cells you may encounter during this course. It's mostly intended as a reference for those interested in the internals of the FPGA. You can find the complete specification [here](#). You should have a (very) basic understanding of their function though, so that you are able to interpret your synthesis results. Sometimes you expect to see a certain cell, but Yosys can't infer it from your design and replaces it with logic.

3.1 SB_LUT4

As mentioned earlier, **SB_LUT4** is a lookup table with 4 inputs. You can think of a truthtable - as it's just that. It's the main building block in our FPGA and most of the logic in your design will be made up of them. They look like this:



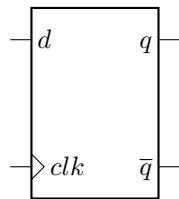
and follow this behavior:

I3	I2	I1	I0	O
0	0	0	0	LUT_INIT[0]
0	0	0	1	LUT_INIT[1]
0	0	1	0	LUT_INIT[2]
0	0	1	1	LUT_INIT[3]
0	1	0	0	LUT_INIT[4]
0	1	0	1	LUT_INIT[5]
0	1	1	0	LUT_INIT[6]
0	1	1	1	LUT_INIT[7]
1	0	0	0	LUT_INIT[8]
1	0	0	1	LUT_INIT[9]
1	0	1	0	LUT_INIT[10]
1	0	1	1	LUT_INIT[11]
1	1	0	0	LUT_INIT[12]
1	1	0	1	LUT_INIT[13]
1	1	1	0	LUT_INIT[14]
1	1	1	1	LUT_INIT[15]

LUT_INIT[15:0] is just a generic parameter that is set, when the cell is placed. It stores the row values of the truthtable. You can see that this cell can implement any truthtable or boolean function with 4 inputs

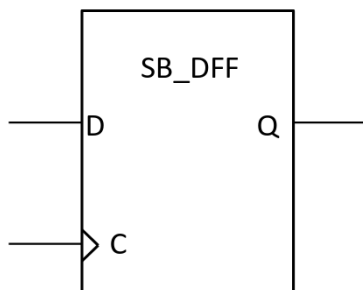
3.2 SB_DFF

Cells with SB_DFF in their name are D flip-flops. Remember them from the lecture:



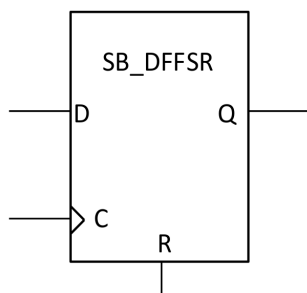
clk	d	q^{t+1}
0/1/↓	—	q^t
↑	1	1
↑	0	0

The iCE40 features different types of these, differing in their clock and reset configuration. If you have a clock in your design and only write to a signal on the rising edge of the clock (or any other condition), then Yosys will infer the corresponding **SB_DFF*** matching your configuration for the signal. For example, this is what the standard DFF with no reset and sensitivity on the posedge looks like:



	clk	d	q
	↑	0	0
	↑	1	1
Power On State	X	X	0

and another variant **SB_DFFSR**, which has a synchronous reset and is sensitive on the posedge of the clk. Here's the block diagram and truthtable:

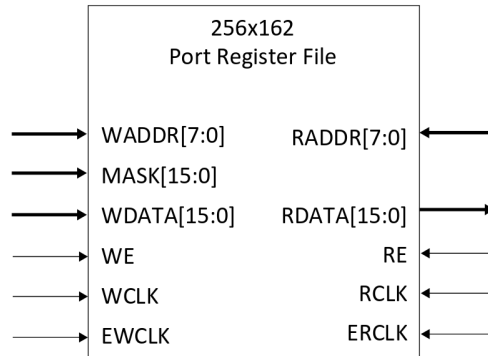


r	clk	d	q
1	X	X	prev q
X	↑	0	0
0	↑	1	1
0	X	X	0
Power On State			

This truthtable is quite unintuitive compared to the lecture. But the synthesis is smart about using the correct one efficiently

3.3 SB_RAM

We could build RAM from flip-flops, wasting lots of them in the process. Our FPGA features special cells for creating RAM, which Yosys will infer for you, if you write your design correctly. They offer different configurations, here is one of them:



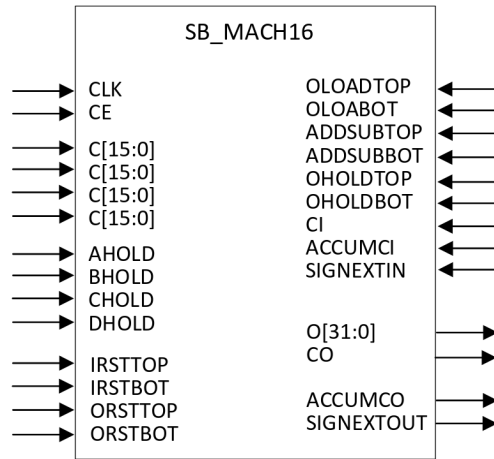
This one is configured to store 256 blocks of 16 bit data. The IO has the following meaning (shorted a bit):

Name	Direction	Purpose
WDATA[15:0]	Input	Write Data input
MASK[15:0]	Input	Bit-line Write Enable input
WADDR[7:0]	Input	Write Address input
WE	Input	Write Enable input
WCLK	Input	Write Clock input
WCLKE	Input	Write Clock Enable input
RDATA[15:0]	Output	Read Data output
RADDR[7:0]	Input	Read Address input
RE	Input	Read Enable input
RCLK	Input	Read Clock input
RCLKE	Input	Read Clock Enable input

Yosys *will* infer the RAM for you, but your design *must* fulfill some criteria. Notably, the **SB_RAM** does not have a reset, so you must not implement a reset logic that expects the entire RAM to set itself to 0 in one cycle. Yosys would infer flipflops and logic from this. Ask us, if you expect to see RAM, but Yosys can't infer it and you don't find the cause. Sometimes it's only subtle.

3.4 SB_MAC16

This cell is a DSP (Digital Signal Processor), which is used to perform wide arithmetic. It can be used to filter and process digitalized analog signals. For this, it can be configured as a multiplier, accumulator, adder, In this course, you will use it to multiply two 32 bit numbers in one clock cycle. Yosys does this automatically, if you use the multiplication operator in SystemVerilog (*). Its interface looks like this:



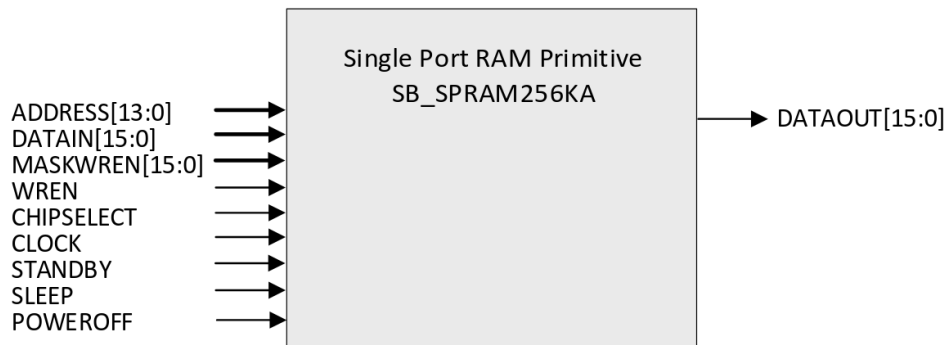
3.5 SB_IO

This cell is added to every IO of your toplevel design that you want to flash on the FPGA. In this course you will likely never need to dive deeper here.

For completeness: SB_IO allows for far more powerful configurations of the pins than simply input or output. We can represent tristate logic (0, 1, and Z for high-impedance) to attach to a system bus or configure them with DDR (double-data-rate). Yosys doesn't support most of those and if it does, it is very picky. You should instantiate the cells manually with your wanted configuration. Please refer to the Lattice datasheet for more information.

3.6 SB_SPRAM

In addition to the RAM from Section 3.3, the FPGA features a second form of RAM: single port RAM. It is less flexible, but suitable for larger memories. It looks like this:



You should try to first exhaust the capabilities presented in Section 3.3 before falling back to this cell. On the paper, Yosys should be able to infer it, but you are better off manually instantiating it in your code, which works perfectly fine but prevents you from simulating your design (the simulator doesn't have a behavior for it). Please ask us, if you want to use it in your design, we will help you get it working. Otherwise refer to the official [Lattice documentation](#).

3.7 SB_CARRY

This cell is mostly used to accelerate arithmetic logic. Yosys does everything for you.

3.8 SB_GB

Global Buffer. Usually used for signals that get distributed to many locations (like a clock)

4 All Makefile targets

This section just lists out all Makefile targets and their purpose. Reminder: You can execute any of them by running `make <...>`. The Makefile knows their dependencies and executes everything for you. Running `make` on its own will execute the entire build process from simulation to flashing the FPGA.

- *simulation*: Simulate the module using the testbench
- *synthesis*: Synthesize for the iCE40 using Yosys
- *synthesis_generic*: Synthesize with logic gates
- *visualize_ice40*: Render the netlist as a PDF
- *visualize_generic*: Render the netlist as a PDF
- *pr*: Perform place & route. Creates an empty .pcf file if none is present
- *pr-gui*: Open the nextpnr gui. Use this if you are curious how your design looks routed. The build process will always use pr
- *gen_bitstream*: Generate the binary bitstream
- *flash_bitstream*: Flashes the bitstream to the FPGA (connect the FPGA first)
- *clean*: Removes all build artifacts